FUNDACIÓN UNIVERSITARIA AGRARIA DE COLOMBIA – UNIAGRARIA
Dirección de Educación Virtual · Ciencia de Datos (Profesional)
Actividad 2 – Hands on: configuración de herramientas
Estudiante: Andres Felipe Amaya Camacho
Docente: Carlos Andres Gomez Vasco

Capítulo 1. Inicio

Control de versiones, historia de Git, instalación y primera configuración

Este capítulo presenta qué es el control de versiones, por qué Git se volvió el estándar de la industria y cómo dejar la herramienta lista para trabajar: instalación, identidad del usuario y consulta de ayuda.

1.1 Acerca del control de versiones

Un sistema de control de versiones (VCS) registra los cambios hechos a un conjunto de archivos a lo largo del tiempo, de modo que es posible volver a una versión anterior, comparar cambios y saber quién modificó qué y cuándo. Es útil para código, pero también para documentos, datos y notebooks de ciencia de datos.

- **Locales:** guardan los cambios en una base de datos en el mismo computador (por ejemplo, RCS). Son simples pero no permiten colaborar.
- **Centralizados (CVCS):** un único servidor guarda todas las versiones y los clientes descargan lo que necesitan (CVS, Subversion). Si el servidor falla, se pierde la historia y la colaboración.
- **Distribuidos (DVCS):** cada persona clona el repositorio completo con toda su historia (Git, Mercurial). Si un servidor falla, cualquier clon sirve para restaurarlo.

1.2 Breve historia de Git

Git nació en 2005, cuando la comunidad del núcleo de Linux dejó de usar la herramienta propietaria BitKeeper y Linus Torvalds diseñó un reemplazo. Sus metas fueron: velocidad, diseño simple, soporte para desarrollo no lineal (miles de ramas paralelas), carácter totalmente distribuido y capacidad de manejar proyectos muy grandes.

1.3 Qué es Git

La diferencia clave de Git frente a otros sistemas es cómo ve los datos: en lugar de guardar listas de diferencias por archivo, almacena **instantáneas (snapshots)** del proyecto completo en cada confirmación. Si un archivo no cambió, simplemente guarda un enlace a la versión idéntica anterior.

- **Casi todas las operaciones son locales:** la historia está en su disco, por lo que consultar o comparar es inmediato y se puede trabajar sin conexión.

- **Integridad:** todo se verifica con una suma de comprobación (hash SHA-1) antes de almacenarse; es imposible alterar un archivo sin que Git lo detecte.
- **Generalmente solo añade datos:** es muy difícil perder información una vez confirmada.

Un archivo puede estar en uno de tres estados: **modificado** (cambió pero no está marcado), **preparado** (staged; marcado para la próxima confirmación) y **confirmado** (guardado de forma segura en la base de datos local). Esto define tres áreas: el directorio de trabajo, el área de preparación (staging area) y el directorio .git.

1.4 La línea de comandos

Existen interfaces gráficas, pero la línea de comandos es la única forma de ejecutar todos los comandos de Git y es la que se usa en este tutorial. En Windows se recomienda Git Bash; en macOS y Linux basta con la terminal.

1.5 Instalación de Git

En Windows se descarga el instalador desde git-scm.com (incluye Git Bash). En macOS puede instalarse con Xcode Command Line Tools o con Homebrew. En Linux se usa el gestor de paquetes de la distribución. Para verificar la instalación:

```
# Debian/Ubuntu
sudo apt install git
# Fedora
sudo dnf install git
# Verificar versión instalada
git --version
```

1.6 Configuración inicial

Git guarda su configuración en tres niveles: `--system` (todos los usuarios), `--global` (el usuario actual) y local (solo el repositorio). Lo primero que debe definirse es la identidad, porque cada commit queda firmado con ese nombre y correo.

```
git config --global user.name "Andres Felipe Amaya Camacho"
git config --global user.email "su_correo@ejemplo.com"
git config --global init.defaultBranch main
git config --global core.editor "code --wait"
# Revisar toda la configuración
git config --list
```

1.7 Obtener ayuda

Git incluye documentación para cada comando:

```
git help <comando>
git <comando> --help
git add -h      # resumen corto en la terminal
```

Conclusión del capítulo

Con Git instalado y configurado, y entendiendo la lógica de instantáneas y los tres estados, ya se puede comenzar a crear y gestionar repositorios, tema del siguiente capítulo.

Fuente: Chacon, S. y Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book/es/v2